



Construcción de software y toma de decisiones (Gpo 501)

Lab 19 Subida y Baja de archivos

Carlos Arturo Gómez Ayala - A01711027

ITESM Campus Querétaro

Profesor: Enrique Alfonso Calderón Balderas

Co-Titular: Alejandro Fernández Vilchis

Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 18/05/2026

¿En qué se diferencia de npm?

Aspecto	npm	pnpm
Almacenamiento	Copia cada dependencia dentro de <code>node_modules</code> de cada proyecto	Guarda una sola copia de cada versión en un almacén global y la enlaza (hard links) a cada proyecto
Espacio en disco	Se duplica en cada proyecto	Se comparte; ahorra mucho espacio
Velocidad	Más lento al reinstalar	Mucho más rápido (reusa el almacén global)
<code>node_modules</code>	Plano: cualquier paquete puede importar dependencias que no declaró	Estricto: solo puedes importar lo que declaraste en <code>package.json</code>
Seguridad	Scripts de instalación se ejecutan por defecto	A partir de pnpm 10 los <i>lifecycle scripts</i> de dependencias no se ejecutan automáticamente ; debes aprobarlos explícitamente

Instalación (una sola vez, de forma global)

`npm install -g pnpm`

Verifica la versión

`pnpm --version`

Despues correr

`pnpm init`

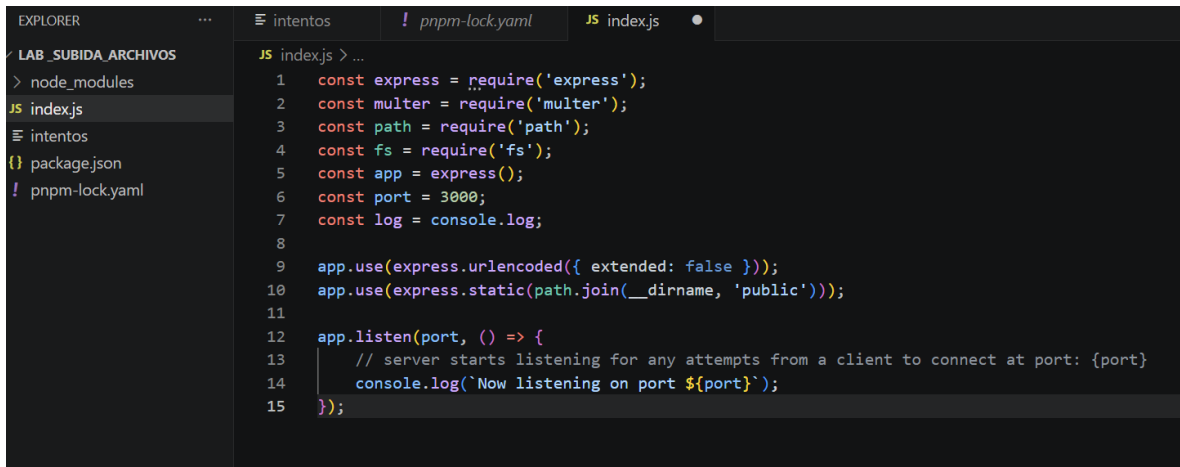
Para obtener el package,json

Se usaran librerias basicas para este laboratorio y estas son

`pnpm add express`

`pnpm add multer`

Se creara un pnm-lock,,, y después de este procederemos a crear index.js y anotar el siguiente código



```
1  const express = require('express');
2  const multer = require('multer');
3  const path = require('path');
4  const fs = require('fs');
5  const app = express();
6  const port = 3000;
7  const log = console.log;
8
9  app.use(express.urlencoded({ extended: false }));
10 app.use(express.static(path.join(__dirname, 'public')));
11
12 app.listen(port, () => {
13   // server starts listening for any attempts from a client to connect at port: {port}
14   console.log(`Now listening on port ${port}`);
15 });
```

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const app = express();
const port = 3000;
const log = console.log;
```

```
app.use(express.urlencoded({ extended: false }));
app.use(express.static(path.join(__dirname, 'public')));
```

```
app.listen(port, () => {
  // server starts listening for any attempts from a client to connect at port: {port}
  console.log(`Now listening on port ${port}`);
});
```

Importante añadir la carpeta de “public” en la carpeta

Despues de esto se hace un cambio de port a 3000 para ejecutar en el servidor

Ya que estamos sirviendo la carpeta pública, vamos a crearla dentro del proyecto y dentro de ella vamos a crear un archivo **index.html**, con el siguiente contenido.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello World Simple App</title>
```

```

</head>
<body>
  <div class="container">
    <h1>Multipart File Upload</h1>
    <form action="/upload_file" method="POST" enctype="multipart/form-data"
id="form">
      <div class="input-group">
        <label for="files">Select files</label>
        <input id="file" name="file" type="file" />
      </div>
      <button class="submit-btn" type="submit">Upload</button>
    </form>
  </div>
</body>
</html>

```

Al trabajar con archivos se requiere tener un formulario como el ejemplo de action que tiene una url un “method”, un multipart y un id, Esto es un **standard**.

Declarar esto es importante

```

action="/upload_file"
method="POST"
enctype="multipart/form-data"

```

El profe Alex recomienda utilizar Dropzone.js <https://www.dropzone.dev/>
Que es para que quede elegante, por tiempo el profesor no lo recomienda, no es un must pero si un plus.

Se creara index.controller.js con el siguiente código

```

const log = console.log

module.exports.upload_file = async (req, res) => {
  log("Cargando el archivo")
  res.status(200).json({code: 200, msg:"Ok"})
}

```

Por último en nuestro archivo **index.js** antes de la declaración del servidor vamos a agregar la ruta para subir la imagen de la siguiente forma.

```
const controller = require("./index.controller.js")
app.post('/upload_file', controller.upload_file);
```

Uter funciona como un híbrido entre multipart, para tomar el multipart de nuestro formulario y recibir el archivo.

dentro de nuestro archivo **index.controller.js** arriba de la función **upload_file**.

```
const multer = require('multer'); // Using Promise API

const storage = multer.diskStorage({
  destination: function (req, file, callback) {
    console.log("File Destination:", './public/'); // Log the destination path
    callback(null, './public/');
  },
  filename: function (req, file, callback) {
    console.log("Uploaded File:", req.body); // Log received form data
    return callback(null, file.originalname);
  }
});

const upload = multer({ storage: storage }).array('file', 1);
```

carga la librería de **multer** y especifica que el destino del archivo sea la carpeta pública que definimos hace unos pasos.

pasaremos el nombre que recibimos desde el inicio.
Ahora dentro de nuestra función de **/upload_file** vamos a sustituir el `res.status(200).json({code: 200, msg:"Ok"})`
el handler completo dentro de **index.controller.js** quede así:

```
module.exports.upload_file = async (req, res) => {
  upload(req, res, function (err) {
    if (err) {
      console.error(err);
      return res.status(500).json({ code: 500, msg: "Error uploading file" });
    }

    console.log("Upload Successful:", req.files); // Log uploaded files
    res.status(200).json({ code: 200, msg: "Ok" });
  });
}
```

```
});  
}
```

La línea más importante de todo este código es la siguiente:

const upload = multer({ storage: storage }).array('file', 1);

Al ser el parámetro de 1, permite la subida de un archivo, le puedes poner otro como un 2, pero solo permite seleccionar un archivo. Para cambiar eso se necesita poner los corchetes de file

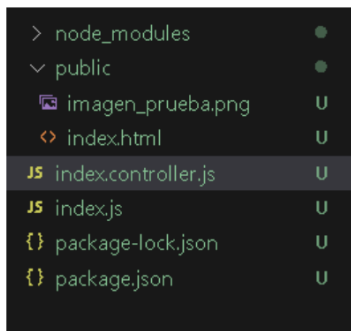
Aquí no solo llamamos a nuestra configuración de **multer**, sino que vamos a recibir un arreglo de archivos que vienen de el **index.html** y el string **'file'** es el id que otorgamos en el formulario al **input** en su propiedad de **name** recuerda siempre esto ya que el primer error que se comete al estar aprendiendo en los formularios es definir estos ids.

Para nuestro caso solo vamos a subir un archivo pero **multer** nos permite agregar múltiples archivos desde la propiedad del file a partir de nuestro form. Te dejo esta configuración en caso de que en otros proyectos quieras trabajar con múltiples archivos, funciona prácticamente igual.

```
var pathDest = req.files[0].destination.slice(1)
```

En esta línea puedes ver como funciona el arreglo puesto que al llamar **req.files[0]** lo que estamos haciendo es llamar al archivo según hayamos subido y aunque sea 1 sabemos que el primero será el de la posición 0.

Nuevamente vamos a ejecutar nuestro servidor y si volvemos a probar el resultado será el mismo pero dentro de nuestro proyecto pasará lo siguiente.



Como puedes observar el archivo que hayamos puesto se ha subido correctamente a nuestra carpeta pública.

Y dentro de nuestro archivo **index.controller.js** definiremos la función:

```
module.exports.upload_file_private = async (req, res) => {  
  log("Cargando el archivo")  
  res.status(200).json({code: 200, msg:"Ok"})  
}
```

Quedando ambos códigos dentro del controller

```
const upload = multer({ storage: storage }).array('file', 1);

module.exports.upload_file_private = async (req, res) => {
  log("Cargando el archivo")
  res.status(200).json({code: 200, msg:"Ok"})
}
```

Agrega esta línea en tu index.js justo después de `const fs = require('fs');`:

```
fs.mkdirSync('./private', { recursive: true });
```

El siguiente paso será crear una nueva configuración de multer para subir nuestros nuevos archivos a esta carpeta.

```
const storage2 = multer.diskStorage({
  destination: function (req, file, callback) {
    callback(null, './private/');
  },
  filename: function (req, file, callback) {
    return callback(null, file.originalname);
  }
});
const upload2 = multer({ storage: storage2 }).array('file', 1);
```

El multer y la practicca son importantes para subir y recibir archivos, se puede poner limites y para mejorar la experiencia de usuario lo mejor es indicarle cuantos archivos puede subir.

Por el código de subida de la información, dejando la función `upload_file_private` así:

```
module.exports.upload_file_private = async (req, res) => {
  upload2(req, res, function (err) {
    if (err) {
      console.error(err);
      return res.status(500).json({ code: 500, msg: "Error uploading file" });
    }
  });
}
```

```
    }

    console.log("Upload Successful:", req.files); // Log uploaded files
    res.status(200).json({ code: 200, msg: "Ok" });
  });
}
```

Esta ruta va dentro de **index.js**:

```
app.get('/get_private_file/:file', controller.get_private_file);
```

Se requiere crear el path

```
const path = require('path');
```

Un extra es saber que los archivos se requieran encriptados, después de verificar que todo funciona se mueve dentro del module.exports